

USING ERROR-CORRECTING CODES TO SIMULATE CHANNELS

A Design Project Report

Presented to the School of Electrical and Computer Engineering
of Cornell University

in Partial Fulfillment of the Requirements for the Degree of

Master of Engineering, Electrical and Computer Engineering

Submitted by

Jaden Zou (jz2396)

MEng Field Advisor: Aaron Wagner

Degree Date: May 2026

Abstract

Master of Engineering Program

School of Electrical and Computer Engineering

Cornell University

Design Project Report

Project Title: Using Error-Correcting Codes to Simulate Channels

Author: Jaden Zou (jz2396)

Abstract: This report documents an M.Eng. design project investigating polar codes over higher-order alphabets. Starting from a theoretical study of polar codes and channel polarization, a complete simulator was implemented and validated first over the binary erasure channel (BEC), then generalized to the binary symmetric channel (BSC) using Monte Carlo sub-channel construction, and subsequently extended to q -ary symmetric channels for $q \in \{2, 3, 5, 7\}$. Subchannel capacities were estimated via Monte Carlo construction, normalized by $\log_2 q$, and plotted together for all four alphabet sizes for a side-by-side comparison, demonstrating that polarization persists and sharpens with increasing q for the tested channel parameters.

Executive Summary

This project investigates whether the polarization phenomenon that makes polar codes capacity-achieving in the binary case persists and sharpens as the alphabet size q grows. The work progressed through three channel models: the binary erasure channel (BEC), the binary symmetric channel (BSC), and the q -ary symmetric channel (qSC) - with each stage building directly on the last.

A polar encoder and LR-based successive cancellation (SC) decoder were implemented and validated over the BEC. The framework was then extended to the BSC using Monte Carlo sub-channel construction, and finally generalized to q -ary symmetric channels by modifying the polar transform, channel model, and decoder kernels accordingly.

BEC and BSC pipelines were validated via BLER vs. K curves, confirming cliff behavior at the correct theoretical capacity in both cases. For the q -ary extension, BLER comparison across alphabet sizes proved difficult to interpret, motivating a shift to normalized sub-channel capacity as the primary metric. Subchannel capacities were plotted together for $q \in \{2, 3, 5, 7\}$ at $N = 256$ and $p = 0.1$, demonstrating that polarization persists and sharpens with increasing q , with the transition from near-zero to near-unity capacity becoming steeper as q grows.

Future work: Extend to larger block lengths, investigate other channels, investigate, build a lower complexity decoder, and more.

Individual Contributions

Since this is a solo project, I am solely responsible for all aspects of this project across both semesters. This includes literature review, theoretical analysis, all code implementation (encoder, channel model, decoder, q -ary extension), experimental design, Monte Carlo simulations, plot generations, debugging, and written documentation including this report and the end-of-semester poster.

Contents

Executive Summary	2
Individual Contributions	3
1 Introduction and Problem Definition	7
1.1 System Requirements	7
2 Background and Theoretical Foundations	8
2.1 Polar Codes and Channel Polarization	8
2.2 Encoder: The Polar Transform	8
2.3 Decoder: Successive Cancellation	10
2.4 Binary Erasure Channel (BEC)	11
2.5 Binary Symmetric Channel (BSC)	11
2.6 q -ary Symmetric Channels	12
3 Design Approach and Methodology	13
4 Semester 1: BEC Baseline Implementation	13
4.1 Literature Review and Theoretical Study	13
4.2 Polar Encoder	14
4.3 BEC Channel Model	14
4.4 Sub-channel Selection	14
4.5 Semester 1 Results	15
5 Semester 2	15
5.1 BEC Decoder Implementation	15
5.2 BEC Results	16
5.3 Transition to the BSC: Monte Carlo Sub-channel Construction	17
5.4 BSC Results	18

5.5	Extension to Higher-Order Alphabets	19
6	q-ary Results	20
6.1	Initial BLER Comparison	20
6.2	Normalized Subchannel Capacity	22
7	Conclusions	24
8	Future Work	24
A	BEC Implementation	27
A.1	Polar Encoder	27
A.2	BEC Channel Model	27
A.3	BEC Sub-channel Selection	28
A.4	LR Kernel Functions	28
A.5	BEC Observation to LR Conversion	29
A.6	SC Decoder	30
B	BSC Implementation (New Functions)	32
B.1	BSC Channel Model	32
B.2	BSC Observation to LR Conversion	32
B.3	Single Sub-channel LR Helper	33
B.4	Monte Carlo Sub-channel Construction	34
C	q-ary Implementation (New Functions)	34
C.1	q-ary Symmetric Channel Model	35
C.2	Channel Transition Probability and Observation Conversion	35
C.3	q-ary Kernel Functions	36
C.4	q-ary SC Decoder	37
C.5	Monte Carlo Sub-channel Construction	39

D Higher Alphabet BLER for $q \in \{2, 3, 4, 5, 6, 7\}$ **40**

1. Introduction and Problem Definition

Error-correcting codes (ECCs) are used to protect information transmitted over noisy channels. By introducing structured redundancy into a message before transmission, a receiver can detect and correct errors introduced by the channel, recovering the original message reliably even when the channel corrupts some of the transmitted symbols.

Polar codes, introduced by Arikan [1], are a particularly significant class of ECC. They are the first provably capacity-achieving codes with efficient encoding and decoding, meaning they can transmit information at the maximum possible rate for a given channel while still allowing reliable recovery. This is achieved through the phenomenon of channel polarization, which is discussed in detail in the background.

Polar codes were originally constructed and analyzed for binary-input channels, where the transmitted symbols are drawn from an alphabet of size $q = 2$. A natural and open question is whether polar codes can be extended to larger alphabets ($q > 2$), and critically, whether the polarization phenomenon that makes them work in the binary case still holds. This project investigates that question. Starting from a binary baseline implementation, the goal is to generalize the polar code construction to higher-order alphabets and study how polarization behavior changes as q grows.

1.1 System Requirements

The key requirements guiding this project were:

- Implement a polar encoder consistent with Arikan’s original construction
- Implement a baseline channel model and validate the full encode/channel/decode pipeline
- Extend the framework to q -ary alphabets and verify that polarization behavior is preserved

- Characterize how polarization changes with alphabet size $q \in \{2, 3, 5, 7\}$ using subchannel capacity as the primary metric

2. Background and Theoretical Foundations

This section provides the theoretical background necessary to understand or reproduce the implementation described in this report. Readers already familiar with polar codes may wish to skip directly to Section 3.

2.1 Polar Codes and Channel Polarization

Polar codes, introduced by Arikan [1], are a class of capacity-achieving error-correcting codes for symmetric binary-input discrete memoryless channels. The central idea is *channel polarization*: by applying a recursive linear transform to N independent uses of a channel, one creates N synthetic sub-channels that progressively polarize—some become nearly perfect (capacity approaching 1) and the rest become nearly useless (capacity approaching 0). As $N \rightarrow \infty$, the fraction of near-perfect sub-channels converges to the capacity of the underlying channel.

In a polar code of block length $N = 2^n$ and rate K/N , the K most reliable sub-channels carry information bits, while the remaining $N - K$ sub-channels are *frozen* to a known value (typically zero). Because the receiver knows which sub-channels are frozen, it can decode the information bits reliably using successive cancellation.

2.2 Encoder: The Polar Transform

The polar transform is defined recursively using the 2×2 kernel

$$F = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}.$$

For block length $N = 2^n$, the generator matrix is $G_N = F^{\otimes n}$ (the n -fold Kronecker power of F), and the encoded codeword is

$$\mathbf{x} = \mathbf{u} \cdot G_N \pmod{2},$$

where $\mathbf{u}$ is the length- N input vector with frozen positions set to zero. The transform is applied recursively, combining pairs of bits at each level, which gives the encoder an $\mathcal{O}(N \log N)$ complexity.

Figure 1 illustrates the encoder structure for $N = 4$. Each U_i is an input bit (information or frozen), each X_i is the corresponding encoded output bit sent over the channel, W represents one use of the underlying channel, and Y_i is the received symbol. The $\oplus$ nodes perform modulo-2 addition, implementing the recursive combining structure of $G_4 = F^{\otimes 2}$. It can be seen that each transmitted bit X_i is a modulo-2 combination of some subset of the inputs $U_1, \dots, U_4$, spreading the information across all channel uses.

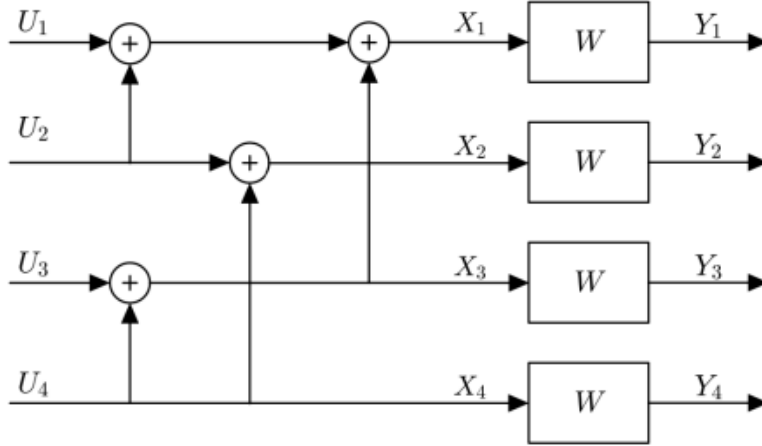


Figure 1: Encoder recursion structure for $N = 4$, showing input bits $U_1, \dots, U_4$, encoded bits $X_1, \dots, X_4$, channel instances W , and received symbols $Y_1, \dots, Y_4$.

2.3 Decoder: Successive Cancellation

The successive cancellation (SC) decoder, as originally formulated by Arikan [1], recovers the input vector $\mathbf{u}$ sequentially from u_0 to u_{N-1} using likelihood ratios (LR). At each step i , the decoder has access to the full channel output $\mathbf{y}$ and all previously decoded bits $u_0, \dots, u_{i-1}$, and computes the LR for u_i defined as

$$L_N^{(i)}(\mathbf{y}, \hat{u}_1^{i-1}) \triangleq \frac{W_N^{(i)}(\mathbf{y}, \hat{u}_1^{i-1} | 0)}{W_N^{(i)}(\mathbf{y}, \hat{u}_1^{i-1} | 1)},$$

where $W_N^{(i)}$ is the transition probability of the i -th synthetic sub-channel. The decision rule is

$$\hat{u}_i = \begin{cases} 0 & \text{if } L_N^{(i)} \geq 1, \\ 1 & \text{otherwise.} \end{cases}$$

If position i is frozen, the decoder sets $\hat{u}_i = 0$ without consulting the LR.

The LR is computed recursively through the polar tree using two kernel functions (equations 75 and 76 in [1]):

$$f(a, b) = \frac{1 + ab}{a + b}, \quad g(a, b, u) = \begin{cases} ab & \text{if } u = 0, \\ b/a & \text{if } u = 1, \end{cases}$$

where a and b are LR values from adjacent channel observations, and u is the previously decoded upper bit. The f function is used for the upper branch of each recursion level before any bits in that half are decoded, and g is used for the lower branch once the upper bits are known. The SC decoder has $\mathcal{O}(N \log N)$ complexity, matching the encoder.

For the q -ary extension, scalar LRs are replaced by length- q probability vectors $[P(y | x = 0), \dots, P(y | x = q-1)]$, and the f and g kernels are generalized to operate over these vectors. The upper branch sums over the unknown lower symbol, and the lower branch conditions

on the already-decoded upper symbol. The final decision at each unfrozen position is the $\arg \max$ of the probability vector.

2.4 Binary Erasure Channel (BEC)

The BEC is the simplest channel model for studying polar codes. Each transmitted bit is received correctly with probability $1 - \varepsilon$, or erased with probability ε . In the implementation here, the erasure symbol is represented as 0.5, since valid bits take only the values 0 or 1.

The BEC is particularly convenient because sub-channel reliability can be tracked *exactly* using the Bhattacharyya parameter $Z(W) \in [0, 1]$, which satisfies a closed-form recursion under the polar transform:

$$Z(W^-) = 2Z(W) - Z(W)^2 \quad (\text{bad synthetic channel}),$$

$$Z(W^+) = Z(W)^2 \quad (\text{good synthetic channel}).$$

This means sub-channels can be ranked analytically without any simulation, making the BEC an ideal baseline for validating encoder and decoder correctness.

2.5 Binary Symmetric Channel (BSC)

The BSC transmits each bit correctly with probability $1 - p$ and flips it with probability p . Unlike the BEC, the BSC does not admit a closed-form polarization recursion. The Bhattacharyya parameter can still be used to bound sub-channel reliability, but it provides only an *upper bound* on the probability of error rather than an exact value. Specifically, the same $Z(W^-)$ and $Z(W^+)$ recursions apply, but $Z(W)$ no longer equals the erasure probability exactly—it is an upper bound on a more complex quantity. As a result, sub-channel selection for the BSC requires Monte Carlo estimation rather than the exact analytic computation available for the BEC.

2.6 q -ary Symmetric Channels

A q -ary symmetric channel (qSC) generalizes the BSC to alphabet size q . Each transmitted symbol is received correctly with probability $1 - p$, and is replaced by each of the other $q - 1$ symbols with equal probability $p/(q - 1)$. For $q = 2$ this reduces exactly to the BSC. The channel capacity is

$$C(q, p) = \log_2 q - H_q(p),$$

where $H_q(p) = -(1 - p) \log_2(1 - p) - p \log_2(p/(q - 1))$ is the q -ary entropy function. For fixed p , capacity increases with q since larger alphabets carry more information per channel use. For fixed q , capacity decreases as p increases, since higher crossover probability means more errors and less reliable transmission, reaching zero when $p = (q - 1)/q$ (the uniform noise case).

A key property of the polar transform is that it preserves total mutual information: the sum of the capacities of all N synthetic sub-channels equals N times the capacity of the underlying channel. That is,

$$\frac{1}{N} \sum_{i=0}^{N-1} C(W^{(i)}) = C(q, p),$$

so the average subchannel capacity equals the channel capacity. Polarization does not create or destroy information; it redistributes it, concentrating it into a fraction $C(q, p)$ of the sub-channels while the rest become nearly useless. As with the BSC, sub-channel reliabilities for the qSC cannot be computed in closed form and must be estimated via Monte Carlo construction.

Note: throughout this report, ε and p are both used to denote the channel error probability (erasure probability for the BEC, crossover probability for the BSC and qSC). The two symbols may sometimes be used interchangeably in the coding context but should not cause any major confusion.

3. Design Approach and Methodology

The overall design strategy was to proceed from simple to complex: validate the binary BEC case completely before extending to more complex channel models. Within each stage, the approach was:

1. Study the theoretical foundation before writing code.
2. Implement incrementally, testing at small block lengths first.
3. Validate behaviour of output against known theoretical predictions (e.g., zero BLER at zero flip probability).
4. Scale up only after small-scale validation passes.

Implementation guidance was drawn from Arikan’s original paper [1], tutorial notes by Pfister [3], and a reference implementation [4] used for structural comparison and validation. Where the reference implementation was consulted this is noted explicitly in the relevant sections below.

4. Semester 1: BEC Baseline Implementation

4.1 Literature Review and Theoretical Study

Prior to any implementation, a significant portion of the first semester was spent developing familiarity with polar codes, as they were not a topic covered in prior coursework. This was done in parallel with ECE 6630 (Information Theory), which meant that I encountered foundational concepts such as mutual information and channel capacity for the first time alongside the project itself.

Key references studied during this period included Arikan’s original paper [1], the tutorial manuscript by Şaşıoğlu [2], and the introductory notes by Pfister [3]. Understanding the polarization phenomenon, synthetic sub-channels, and the successive cancellation decoder

well enough to implement them took the majority of the first semester, with implementation beginning only once the theoretical foundation was sufficiently solid.

4.2 Polar Encoder

The encoder implements the polar transform recursively following the structure introduced in background. The implementation uses an odd/even interleaving structure, where at each recursion level the even-indexed inputs are XOR'd with the odd-indexed ones to form the upper half of the output, and the odd-indexed inputs pass through unchanged to the lower half. Initial validation was performed at $N = 8$ and 16 by comparing them against manually computed vector $\mathbf{x} = \mathbf{u} \cdot G_8 \pmod{2}$ for randomly generated input vectors.

A recursive structure was used, following the implementation approach of Pfister [4], achieves $\mathcal{O}(N \log N)$ encoding complexity, compared to the $\mathcal{O}(N^2)$ complexity of a naive matrix multiplication $\mathbf{u} \cdot G_N$.

The full encoder implementation is listed in Appendix A.1.

4.3 BEC Channel Model

The BEC model independently erases each bit of the transmitted codeword with probability ε . Erased positions are marked with the value 0.5 , chosen because valid bits take only the values 0 or 1 , making erasures unambiguous to the decoder.

The full implementation is listed in Appendix A.2.

4.4 Sub-channel Selection

For the BEC, sub-channel reliability is computed analytically using the Bhattacharyya recursion described in the background section, without any Monte Carlo simulation. The recursion is initialized at $Z = \varepsilon$ and expanded level by level until all N sub-channel parameters are computed. The K smallest values of Z correspond to the K most reliable sub-channels and

are left unfrozen for information bits; the remaining $N - K$ are frozen to zero.

The full implementations of `bec_subchannels` and `freeze_bad_channels` are listed in Appendix A.3.

4.5 Semester 1 Results

Validation was performed at small block lengths ($N = 8$ and $N = 16$). The Bhattacharyya parameters computed by the recursion were verified by hand-tracing the $Z(W^-)$ and $Z(W^+)$ recursion for small N and comparing against the code output. The encoder was similarly validated by manually computing $\mathbf{u} \cdot G_8 \pmod{2}$ for randomly generated input vectors and confirming the code matched. For the BEC channel model, correctness was verified empirically by passing a large vector through the channel and confirming that the fraction of erased positions (marked 0.5) matched the expected erasure probability ε within statistical noise. These results provided a validated baseline pipeline as the foundation for semester 2.

5. Semester 2

This section documents the implementation steps taken during the second semester. The BEC and BSC results are presented inline as intermediate validation steps, since each stage directly motivated the next. The q -ary symmetric channel results, which represent the primary investigation of this project, are presented separately.

5.1 BEC Decoder Implementation

The decoder was partially constructed during the fall semester, but the full implementation was not ready due to many coding bugs during the process. With the encoder and channel model validated, the SC decoder was implemented for the BEC following the LR-based decoding. The f and g kernel functions operate on LR values, where a received 0 maps to $\text{LR} = \infty$, a received 1 maps to $\text{LR} = 0$, and an erasure maps to $\text{LR} = 1$. The decoder

recurses over the polar tree, computing upper branch LRs via f and lower branch LRs via g , making a hard decision at each unfrozen leaf by thresholding at 1.0. The functions required to implement this is listed in Appendix A.4, A.5, and A.6.

5.2 BEC Results

With the decoder debugged and the full pipeline operational, Monte Carlo experiments were run to validate polarization behavior. Figure 2 shows BLER as a function of the number of information bits K .

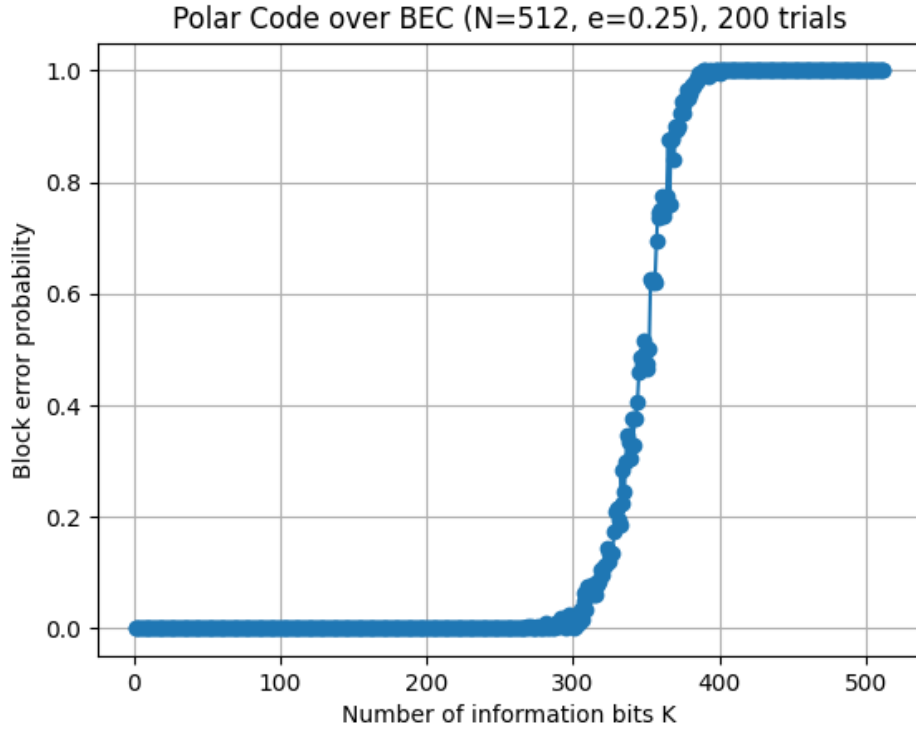


Figure 2: BLER vs. number of information bits K for a polar code over the BEC ($N = 512$, $\varepsilon = 0.25$, 200 trials).

The expected capacity of this BEC is:

$$C = 1 - \varepsilon = 1 - 0.25 = 0.75,$$

meaning reliable transmission is theoretically possible for up to $K = N \cdot C = 512 \times 0.75 = 384$ information bits. The plot shows the characteristic polarization behavior: BLER remains near zero for small K and rises sharply before reaching $K = 384$. The transition occurring slightly before the theoretical limit is expected at finite block lengths. Perfect polarization is only achieved asymptotically as $N \rightarrow \infty$, so at $N = 512$ some sub-channels near the capacity boundary are not yet fully polarized, causing the cliff to appear slightly early. Larger values of N would sharpen and shift this transition closer to the theoretical limit, but at the cost of increased computation time for both the Monte Carlo sub-channel construction and the recursive SC decoder.

To put this in practical terms, the result demonstrates a clear rate–reliability tradeoff: at $K = 300$ information bits (a rate of $300/512 \approx 0.59$), the code achieves near-zero BLER, meaning data can be transmitted at roughly 59% of the channel’s raw capacity with negligible error probability. This is the fundamental promise of polar codes: reliable communication at rates approaching capacity. This result confirms that the implementation is functioning correctly.

5.3 Transition to the BSC: Monte Carlo Sub-channel Construction

With the BEC pipeline fully validated, I moved to working with the BSC case. As discussed the background, sub-channel selection for the BSC requires Monte Carlo estimation rather than the exact Bhattacharyya recursion used for the BEC. The function `bsc_subchannels_monte_carlo` estimates the reliability of each sub-channel i by transmitting random code-words through the BSC and measuring the empirical SC decision error rate for that sub-channel, assuming all previous bits $u_0, \dots, u_{i-1}$ are known and correct. This isolates the error probability of each sub-channel independently, avoiding contamination from decoding errors in earlier positions. A separate helper function `sc_bit_lr` was implemented to compute the LR for a single sub-channel given the previous true bits, which is called inside the Monte Carlo loop for each trial.

The BSC channel model and `bsc_obs_to_lr` were updated to reflect BSC transition probabilities: a received 0 maps to $LR = (1 - p)/p$ and a received 1 maps to $LR = p/(1 - p)$.

The relevant new functions are listed in Appendix B.

5.4 BSC Results

Figure 3 shows BLER as a function of K for the BSC.

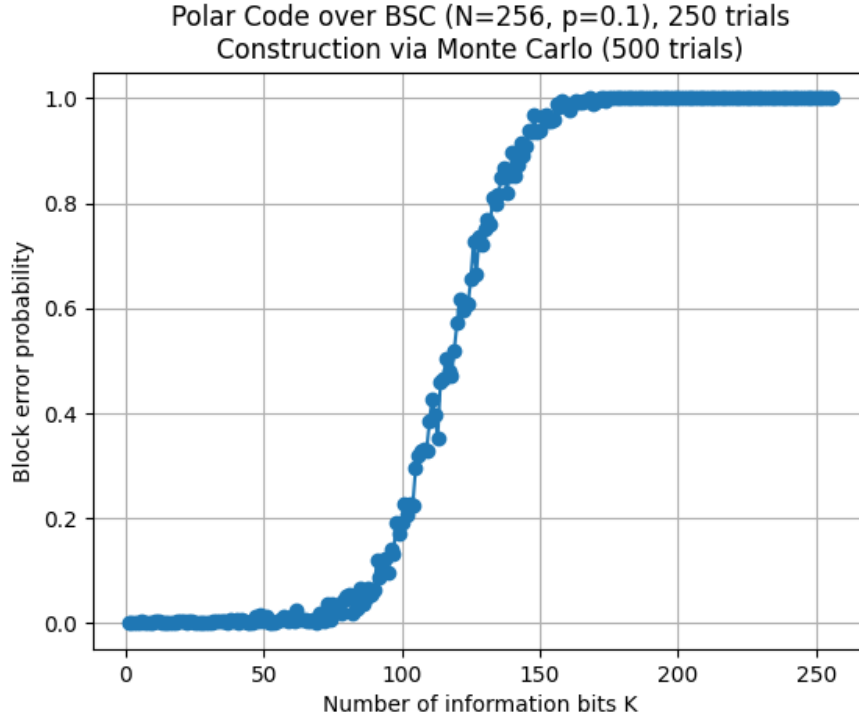


Figure 3: BLER vs. number of information bits K for a polar code over the BSC ($N = 256$, $p = 0.1$, 250 trials, construction via 500 Monte Carlo trials).

The expected capacity of this BSC is

$$C = 1 - H_2(p) = 1 - H_2(0.1) = 1 - 0.469 = 0.531,$$

meaning reliable transmission is theoretically possible for up to $K = N \cdot C = 256 \times 0.531 \approx 136$ information bits. The plot shows the same characteristic cliff behavior seen in the BEC case, with BLER rising sharply at about $K = 80$, and for the same reason.

5.5 Extension to Higher-Order Alphabets

With a working BSC pipeline, the framework was generalized to q -ary symmetric channels for $q \in \{2, 3, 5, 7\}$. This required changes to three components:

Polar transform: The modulo-2 addition (binary XOR operation) was replaced with modulo- q addition. The recursive combining step becomes

$$x[: N/2] = \text{polar_transform}((u[: 2] + u[1 :: 2]) \bmod q),$$

with the recursive structure and $\mathcal{O}(N \log N)$ complexity otherwise unchanged.

Channel model: The BSC is replaced by the qSC. Figure 4 shows the transition diagram for the ternary case ($q = 3$) as an example, illustrating how each input symbol is received correctly with probability $1 - \varepsilon$ and flips to either of the other two symbols with equal probability $\varepsilon/2$.

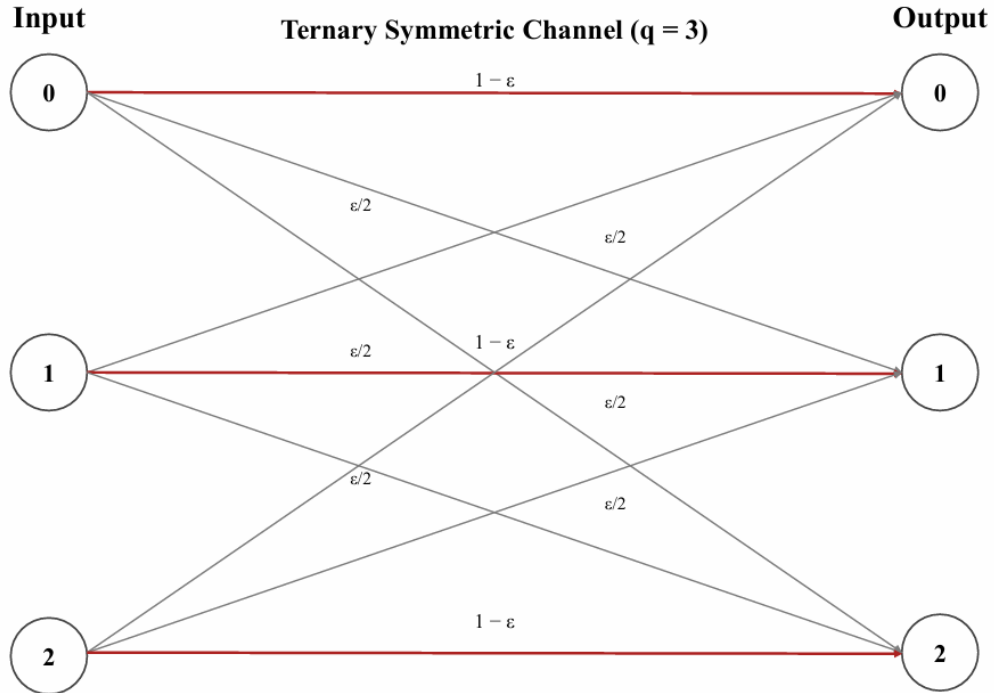


Figure 4: Transition diagram for the ternary symmetric channel ($q = 3$).

Decoder: The most significant change is in how the decoder processes channel observations. For the binary case, each received symbol produces a single scalar LR value, and decisions are made by thresholding at 1.0. For the q -ary case, each received symbol instead produces a length- q probability vector, where entry a represents $P(y_i | x = a)$. The f and g kernel functions are generalized to operate over these vectors rather than scalars, and the final decision at each unfrozen sub-channel is made by taking the arg max of the output probability vector rather than a threshold comparison. Sub-channel construction uses the same Monte Carlo approach as the BSC, now measuring symbol error rate rather than bit error rate. The computational cost of the decoder increases from $\mathcal{O}(N \log N)$ to $\mathcal{O}(q^2 N \log N)$, since each kernel operation now involves summing over all q symbol combinations rather than a single scalar computation. The code for q -ary implementation is listed in Appendix C.

6. q -ary Results

6.1 Initial BLER Comparison

First, the q -ary code was tested the $q = 2$ case to confirm that it presents the same results as the previous BSC implementation shown in Figure 3. Then, with the q -ary pipeline implementation verified, the first approach to comparing polarization behavior across alphabet sizes was to plot BLER vs. K for each value of q , following the same validation approach used for the BEC and BSC. This was done to verify the behavior and functionality of the implementation since many things were changed when moving from a BSC to the qSC; this was done for $q \in \{2, 3, 4, 5, 6, 7\}$. Figure 5 shows a representative example at $q = 5$, which produces a clean cliff consistent with the expected capacity, confirming that the q -ary pipeline is functioning correctly. The individual BLER plots for $q \in \{2, 3, 4, 5, 6, 7\}$ are provided in Appendix D for reference.

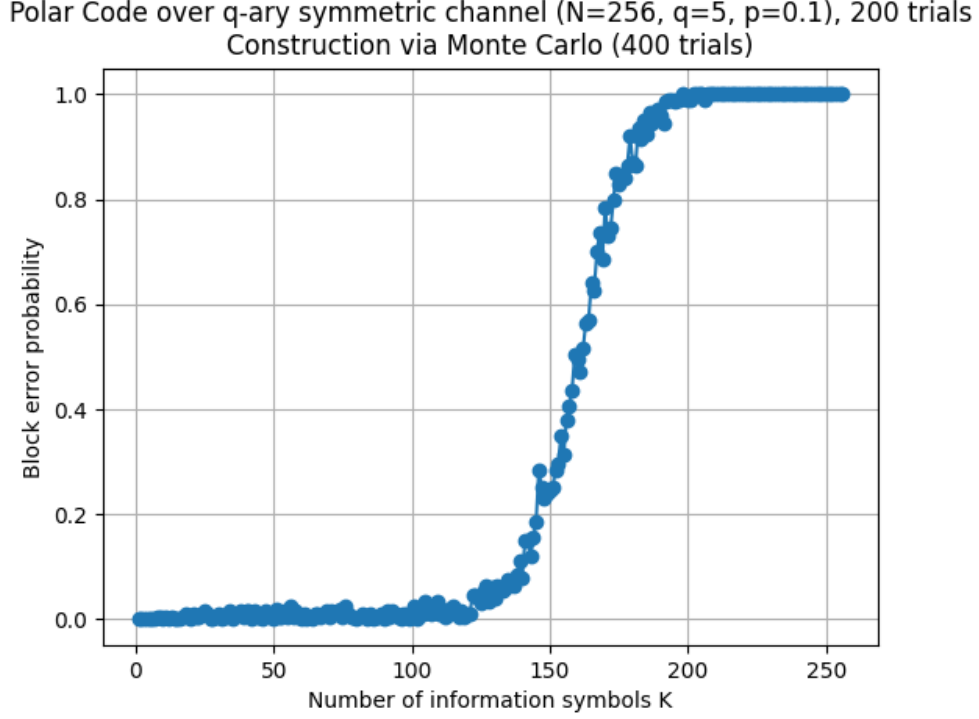


Figure 5: BLER vs. number of information symbols K for a polar code over the qSC ($N = 256$, $q = 5$, $p = 0.1$, 200 trials, subchannel-capacity construction via 400 Monte Carlo trials).

The expected capacity for this channel can be calculated as

$$C(q, p) = \log_2 q - H_q(p) = \log_2 5 - H_5(0.1),$$

where

$$H_5(0.1) = -(1 - 0.1) \log_2(0.9) - 0.1 \log_2\left(\frac{0.1}{4}\right) = 0.137 + 0.332 = 0.469,$$

giving $C(5, 0.1) \approx 2.322 - 0.469 = 1.853$ bits per symbol, or a normalized capacity of $1.853 / \log_2 5 \approx 0.798$. This means reliable transmission should be possible for up to $K \approx 0.798 \times 256 \approx 204$ information symbols,

However, after discussion with the project advisor, it became clear that BLER vs. K is not a meaningful metric for comparing polarization behavior *across* different alphabet sizes. The

issue is that K here counts information *symbols*, not bits. At alphabet size q , each symbol carries $\log_2 q$ bits of information, so a fixed K in symbols corresponds to $K \log_2 q$ bits — more information for larger q . This means larger alphabets are inherently penalized in a BLER vs. K comparison at fixed K , making it impossible to draw conclusions about relative polarization strength.

6.2 Normalized Subchannel Capacity

To enable a fair comparison across alphabet sizes, the metric was changed to normalized subchannel capacity. For each q , the capacity of each synthetic sub-channel is estimated via Monte Carlo construction and then normalized by $\log_2 q$, placing all alphabet sizes on a common $[0, 1]$ scale. Sub-channels are then sorted by their normalized capacity and plotted together. On this scale, a perfectly polarized code would show all sub-channels at either exactly 0 or exactly 1, with the fraction at 1 equal to the normalized channel capacity $C(q, p)/\log_2 q$.

Additionally, following guidance from the project advisor, the comparison was restricted to prime alphabet sizes $q \in \{2, 3, 5, 7\}$. For non-prime q such as 4 or 6, it is hypothesized that the polarization structure differs, and the polarization guarantees may not hold in the same way.

Figure 6 shows the normalized subchannel capacity curves for all four alphabet sizes.

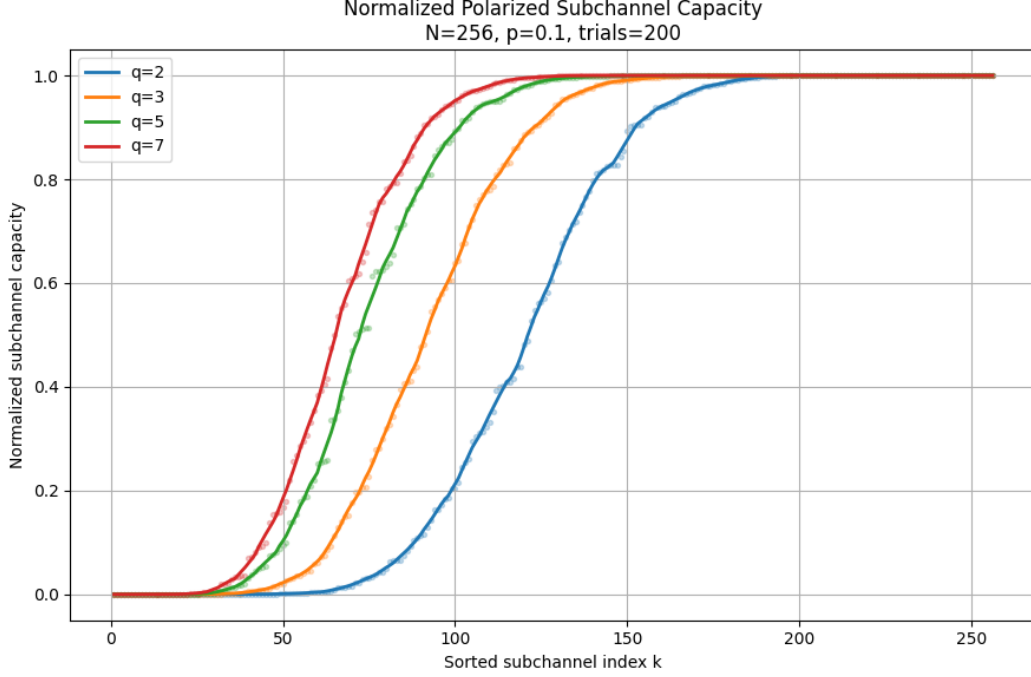


Figure 6: Normalized subchannel capacity vs. sorted subchannel index k for $q \in \{2, 3, 5, 7\}$ ($N = 256$, $p = 0.1$, 200 trials). Sub-channels are sorted in increasing order of capacity.

The plot demonstrates some clear trends:

Polarization persists: For all tested prime alphabet sizes, sub-channels separate into groups of near-zero and near-unity normalized capacity, confirming that the polarization phenomenon extends beyond the binary case.

Transition shifts leftward with increasing q : The cliff position moves left as q grows, reflecting the higher normalized channel capacity. A larger fraction of sub-channels are good, consistent with the capacity formula $C(q, p) = \log_2 q - H_q(p)$.

Polarization sharpens with increasing q : The transition from 0 to 1 becomes steeper and occurs over a narrower range of sub-channel indices as q grows, indicating that larger alphabets polarize more aggressively at this block length. This was an unexpected result.

7. Conclusions

This project successfully demonstrated that polar codes can be constructed and simulated over q -ary symmetric channels for prime alphabet sizes $q \in \{2, 3, 5, 7\}$, and that polarization behavior does indeed occur, increasing in average channel capacity with increasing alphabet size q . The primary result is the normalized subchannel capacity plot for $q \in \{2, 3, 5, 7\}$, which shows that polarization not only persists at higher alphabets but sharpens with increasing q - an unexpected finding. The leftward shift of the capacity transition with increasing q is consistent with the higher channel capacity at larger alphabets, and the steepening of the transition suggests that higher-order alphabets may be inherently better suited to polar coding at finite block lengths, but is not a definitive conclusion. The behavior is not completely understood yet, and further work must still be done to fully understand how speed of polarization occurs.

8. Future Work

- **Polarization behavior at different block lengths:** The sharpening of polarization with q was observed at $N = 256$. It would be valuable to investigate whether this trend holds at larger and smaller N , and how the rate of polarization scales with both q and N .
- **Lower complexity decoding:** The current SC decoder has $\mathcal{O}(q^2 N \log N)$ complexity, which becomes costly at large q . Investigating lower complexity alternatives, such as list decoding or belief propagation adapted to q -ary alphabets, would improve the practical scalability of the implementation.
- **Theoretical analysis of sharpening:** The observation that larger alphabets produce sharper normalized polarization curves was unexpected and warrants further theoretical investigation. Understanding why this occurs could provide insight into the optimal

choice of alphabet size for a given channel and block length.

- **More methods of normalized comparison:** Normalized subchannel capacity is one way to compare polarization across alphabet sizes, but other more robust ways of measuring the speed of polarization could be deployed could be implemented.
- **Alternative channel models:** The current implementation is restricted to symmetric channels. Extending to asymmetric channels or channels with memory would broaden the applicability of the framework.
- **Larger non-prime alphabet sizes:** The current implementation uses modulo- q for prime q . Extending to non-prime alphabets such as $q = 4$ would allow a more complete picture of how polarization behaves across all alphabet sizes.

References

- [1] E. Arıkan, “Channel polarization: A method for constructing capacity-achieving codes for symmetric binary-input memoryless channels,” *IEEE Transactions on Information Theory*, vol. 55, no. 7, pp. 3051–3073, 2009.
- [2] E. Şaşoğlu, “Polarization and polar codes,” 2011. Tutorial manuscript.
- [3] H. D. Pfister, “A brief introduction to polar codes.” Tutorial notes.
- [4] H. D. Pfister, “Polar codes: Introductory implementation.” https://github.com/henrypfister/polar_intro. Accessed December 2025.
- [5] Claude (claude-sonnet-4-20250514, Anthropic), used for debugging assistance, implementation review, and report drafting assistance. Conversations accessed via <https://claude.ai>, Spring 2026.

A. BEC Implementation

A.1 Polar Encoder

```

1 def polar_transform(u):
2     if len(u) == 1:
3         return u
4     else:
5         x = np.zeros(len(u), dtype=np.int64)
6         x[:len(u)//2] = polar_transform((u[:2] + u[1:2]) % 2)
7         x[len(u)//2:] = polar_transform(u[1:2])
8         return x

```

Listing 1: Polar transform encoder.

A.2 BEC Channel Model

```

1 def bec(x, e):
2     r = np.random.random_sample(len(x))
3     y = np.zeros(len(x), dtype=np.float64)
4     for i in range(len(x)):
5         if r[i] < e:
6             y[i] = 0.5
7         else:
8             y[i] = x[i]
9     return y

```

Listing 2: Binary erasure channel model.

Note that the BEC `f` and `g` implementations include additional guards for edge cases involving `inf` and `0`, which arise because BEC likelihood ratios take values in $\{0, 1, \infty\}$. The

BSC versions reduce to simple one-liners since BSC likelihood ratios are always finite and positive.

A.3 BEC Sub-channel Selection

```

1 def bec_subchannels(N, e):
2     Z = np.array([e], dtype=np.float64)
3     while len(Z) < N:
4         Z_new = np.zeros(2 * len(Z), dtype=np.float64)
5         for i in range(len(Z)):
6             Z_new[2*i] = 2*Z[i] - Z[i]**2    # bad channel
7             Z_new[2*i + 1] = Z[i]**2        # good channel
8         Z = Z_new
9     return Z
10
11 def freeze_bad_channels(Z, K):
12     n = len(Z)
13     frozen = np.ones(n, dtype=bool)
14     info_indices = np.argsort(Z)[:K]
15     frozen[info_indices] = False
16     return frozen

```

Listing 3: Bhattacharyya recursion for sub-channel ranking.

A.4 LR Kernel Functions

```

1 def f(a, b):
2     # Arikan LR upper combine (equation 75)
3     # Inputs a, b are likelihood ratios in [0, inf].
4     # 0 = certain 1, inf = certain 0, 1 = erasure
5     if np.isinf(a) and np.isinf(b):

```

```

6         return np.inf
7     if a == 0.0 and b == 0.0:
8         return np.inf
9     if (np.isinf(a) and b == 0.0) or (a == 0.0 and np.isinf(b)):
10        return 0.0
11    if a == 1.0 or b == 1.0:
12        return 1.0
13    return (1.0 + a * b) / (a + b)
14
15 def g(a, b, u):
16     # Arikan LR lower combine (equation 76)
17     if u == 0:
18         if (np.isinf(a) and b == 0.0) or (a == 0.0 and np.isinf(b)):
19             return 1.0 # treat as erasure
20         return b * a
21     else:
22         if a == 0.0:
23             return np.inf # b / 0 -> inf (certain 0)
24         if np.isinf(a) and np.isinf(b):
25             return 1.0 # inf / inf -> erasure
26         return b / a

```

Listing 4: f and g kernel functions (equations 75 and 76 in [1]).

A.5 BEC Observation to LR Conversion

```

1 def bec_obs_to_lr(y):
2     # y = 0    -> LR = inf (definitely x=0)
3     # y = 1    -> LR = 0   (definitely x=1)
4     # y = 0.5 -> LR = 1   (erasure, equal likelihood)

```

```

5     y = np.asarray(y, dtype=np.float64)
6     lr = np.empty_like(y, dtype=np.float64)
7     for i in range(len(y)):
8         if y[i] == 0.0:
9             lr[i] = np.inf
10        elif y[i] == 1.0:
11            lr[i] = 0.0
12        else:
13            lr[i] = 1.0
14    return lr

```

Listing 5: Convert BEC channel outputs to likelihood ratios.

A.6 SC Decoder

```

1 def sc_decode_lr(lr_vec, frozen):
2     def _decode_node(obs, frozen_node):
3         N = len(obs)
4         if N == 1:
5             if frozen_node[0]:
6                 bit = 0
7             else:
8                 bit = 0 if obs[0] >= 1.0 else 1
9             u_hat_node = np.array([bit], dtype=np.int64)
10            x_hat_node = np.array([bit], dtype=np.int64)
11            return u_hat_node, x_hat_node
12
13        half = N // 2
14
15        upper_obs = np.zeros(half, dtype=np.float64)

```

```

16     for i in range(half):
17         upper_obs[i] = f(obs[2*i], obs[2*i+1])
18     u_hat_upper, x_hat_upper = _decode_node(upper_obs,
19         frozen_node[:half])
20
21     lower_obs = np.zeros(half, dtype=np.float64)
22     for i in range(half):
23         lower_obs[i] = g(obs[2*i], obs[2*i+1], x_hat_upper[i])
24     u_hat_lower, x_hat_lower = _decode_node(lower_obs,
25         frozen_node[half:])
26
27     u_hat_node = np.zeros(N, dtype=np.int64)
28     u_hat_node[:half] = u_hat_upper
29     u_hat_node[half:] = u_hat_lower
30
31     x_hat_node = np.zeros(N, dtype=np.int64)
32     for i in range(half):
33         x_hat_node[2*i] = (x_hat_upper[i] + x_hat_lower[i]) %
34             2
35         x_hat_node[2*i+1] = x_hat_lower[i]
36
37     return u_hat_node, x_hat_node
38
39 u_hat, _ = _decode_node(np.asarray(lr_vec, dtype=np.float64),
40     np.asarray(frozen, dtype=bool))
41
42 return u_hat

```

Listing 6: LR-based successive cancellation decoder.

B. BSC Implementation (New Functions)

Functions shared with the BEC implementation (`polar_transform`, `freeze_bad_channels`, `f`, `g`, `sc_decode_lr`) are identical to those in Appendix A and are not repeated here.

B.1 BSC Channel Model

```

1 def bsc(x, p, rng=None):
2     x = np.asarray(x, dtype=np.int64)
3     if rng is None:
4         r = np.random.random_sample(len(x))
5     else:
6         r = rng.random(len(x))
7     y = np.array(x, copy=True, dtype=np.int64)
8     y[r < p] ^= 1
9     return y

```

Listing 7: Binary symmetric channel model.

B.2 BSC Observation to LR Conversion

```

1 def bsc_obs_to_lr(y, p):
2     # LR = P(y | x=0) / P(y | x=1)
3     y = np.asarray(y, dtype=np.int64)
4     lr = np.empty(len(y), dtype=np.float64)
5     if p == 0.0:
6         for i in range(len(y)):
7             lr[i] = np.inf if y[i] == 0 else 0.0
8         return lr
9     lr_y0 = (1.0 - p) / p
10    lr_y1 = p / (1.0 - p)

```

```

11     for i in range(len(y)):
12         lr[i] = lr_y0 if y[i] == 0 else lr_y1
13     return lr

```

Listing 8: Convert BSC channel outputs to likelihood ratios.

B.3 Single Sub-channel LR Helper

```

1 def sc_bit_lr(obs, u_prev, bit_index):
2     obs = np.asarray(obs, dtype=np.float64)
3     u_prev = np.asarray(u_prev, dtype=np.int64)
4
5     def _bit_lr(obs_node, prev_node, target):
6         n = len(obs_node)
7         if n == 1:
8             return obs_node[0]
9         half = n // 2
10        upper_obs = np.zeros(half, dtype=np.float64)
11        for i in range(half):
12            upper_obs[i] = f(obs_node[2*i], obs_node[2*i+1])
13        if target < half:
14            return _bit_lr(upper_obs, prev_node, target)
15        u_upper = prev_node[:half]
16        x_upper = polar_transform(u_upper)
17        lower_obs = np.zeros(half, dtype=np.float64)
18        for i in range(half):
19            lower_obs[i] = g(obs_node[2*i], obs_node[2*i+1], x_upper
20                               [i])
21        return _bit_lr(lower_obs, prev_node[half:], target - half)

```

```
22     return _bit_lr(obs, u_prev, bit_index)
```

Listing 9: Compute the LR for a single sub-channel i given true previous bits, used inside the Monte Carlo construction loop.

B.4 Monte Carlo Sub-channel Construction

```
1 def bsc_subchannels_monte_carlo(N, p, trials, seed=None):
2     rng = np.random.default_rng(seed)
3     error_probs = np.zeros(N, dtype=np.float64)
4     for bit_index in range(N):
5         errors = 0
6         for _ in range(trials):
7             u = rng.integers(0, 2, N, dtype=np.int64)
8             x = polar_transform(u)
9             y = bsc(x, p, rng=rng)
10            lr_vec = bsc_obs_to_lr(y, p)
11            lr_bit = sc_bit_lr(lr_vec, u[:bit_index], bit_index)
12            u_decision = 0 if lr_bit >= 1.0 else 1
13            if u_decision != u[bit_index]:
14                errors += 1
15            error_probs[bit_index] = errors / trials
16    return error_probs
```

Listing 10: Monte Carlo estimation of sub-channel error probabilities for the BSC.

C. q-ary Implementation (New Functions)

Functions shared with previous implementations (`polar_transform`, `freeze_bad_channels`) are not repeated here. Note that the q-ary `f` and `g` kernels operate on length- q probability vectors

rather than scalar LRs, and so are meaningfully different from their binary counterparts in Appendix A.

C.1 q-ary Symmetric Channel Model

```

1 def qary_channel(x, p, q):
2     x = np.asarray(x, dtype=np.int64)
3     r = np.random.random_sample(len(x))
4     e = np.random.randint(1, q, size=len(x))
5     y = np.array(x, copy=True, dtype=np.int64)
6     flip = r < p
7     y[flip] = (y[flip] + e[flip]) % q
8     return y

```

Listing 11: q-ary symmetric channel model.

C.2 Channel Transition Probability and Observation Conversion

```

1 def channel_prob(y, x, p, q):
2     # W(y|x): transition probability of the q-ary symmetric channel
3     if y == x:
4         return 1 - p
5     else:
6         return p / (q - 1)
7
8 def normalize_probs(v):
9     # Rescales probability vector for numerical stability.
10    # Does not change the argmax.
11    s = np.sum(v)
12    if s == 0:
13        return np.ones(len(v), dtype=np.float64) / len(v)

```

```

14     return v / s
15
16 def qary_obs_to_probs(y, p, q):
17     # For each received symbol y[i], compute:
18     # [P(y[i]|x=0), P(y[i]|x=1), ..., P(y[i]|x=q-1)]
19     y = np.asarray(y, dtype=np.int64)
20     probs = np.zeros((len(y), q), dtype=np.float64)
21     for i in range(len(y)):
22         for a in range(q):
23             probs[i, a] = channel_prob(y[i], a, p, q)
24         probs[i] = normalize_probs(probs[i])
25     return probs

```

Listing 12: Channel transition probability and conversion of received symbols to probability vectors.

C.3 q-ary Kernel Functions

```

1 def qary_f(left_probs, right_probs, q):
2     # Upper branch update: sums over the unknown lower symbol.
3     out = np.zeros(q, dtype=np.float64)
4     for a in range(q):
5         total = 0.0
6         for b in range(q):
7             total += left_probs[(a + b) % q] * right_probs[b]
8         out[a] = total / q
9     return normalize_probs(out)
10
11 def qary_g(left_probs, right_probs, upper_symbol, q):

```

```

12     # Lower branch update: conditions on the already-decoded upper
        symbol.
13     out = np.zeros(q, dtype=np.float64)
14     for b in range(q):
15         out[b] = left_probs[(upper_symbol + b) % q] * right_probs[b]
16     return normalize_probs(out)

```

Listing 13: q -ary generalizations of the f and g kernel functions, operating over length- q probability vectors.

C.4 q -ary SC Decoder

```

1 def qary_sc_decode(y, frozen, N, p, q, frozen_values=None):
2     obs_probs = qary_obs_to_probs(y, p, q)
3     frozen = np.asarray(frozen, dtype=bool)
4     if frozen_values is None:
5         frozen_values = np.zeros(N, dtype=np.int64)
6     else:
7         frozen_values = np.asarray(frozen_values, dtype=np.int64)
8
9     def _decode_node(obs_node, frozen_node, frozen_values_node):
10         n = len(obs_node)
11         if n == 1:
12             if frozen_node[0]:
13                 symbol = frozen_values_node[0]
14             else:
15                 symbol = np.argmax(obs_node[0])
16         return (np.array([symbol], dtype=np.int64),
17                np.array([symbol], dtype=np.int64))
18

```

```

19     half = n // 2
20
21     upper_obs = np.zeros((half, q), dtype=np.float64)
22     for i in range(half):
23         upper_obs[i] = qary_f(obs_node[2*i], obs_node[2*i+1], q)
24     u_hat_upper, x_hat_upper = _decode_node(
25         upper_obs, frozen_node[:half], frozen_values_node[:half
26         ])
27
28     lower_obs = np.zeros((half, q), dtype=np.float64)
29     for i in range(half):
30         lower_obs[i] = qary_g(obs_node[2*i], obs_node[2*i+1],
31                                x_hat_upper[i], q)
32     u_hat_lower, x_hat_lower = _decode_node(
33         lower_obs, frozen_node[half:], frozen_values_node[half
34         :])
35
36     u_hat_node = np.zeros(n, dtype=np.int64)
37     u_hat_node[:half] = u_hat_upper
38     u_hat_node[half:] = u_hat_lower
39
40     x_hat_node = np.zeros(n, dtype=np.int64)
41     for i in range(half):
42         x_hat_node[2*i] = (x_hat_upper[i] + x_hat_lower[i]) %
43         q
44         x_hat_node[2*i+1] = x_hat_lower[i]
45
46     return u_hat_node, x_hat_node

```

```

45     u_hat, _ = _decode_node(obs_probs, frozen, frozen_values)
46     return u_hat

```

Listing 14: q-ary successive cancellation decoder using probability vectors.

C.5 Monte Carlo Sub-channel Construction

```

1  def qary_subchannels_monte_carlo(N, p, q, trials):
2      error_probs = np.zeros(N, dtype=np.float64)
3      for symbol_index in range(N):
4          errors = 0
5          frozen = np.ones(N, dtype=bool)
6          frozen[symbol_index] = False
7          for _ in range(trials):
8              u = np.random.randint(0, q, size=N, dtype=np.int64)
9              x = polar_transform(u, q)
10             y = qary_channel(x, p, q)
11             frozen_values = np.zeros(N, dtype=np.int64)
12             frozen_values[:symbol_index] = u[:symbol_index]
13             u_hat = qary_sc_decode(y, frozen, N, p, q,
14                                   frozen_values=frozen_values)
15             if u_hat[symbol_index] != u[symbol_index]:
16                 errors += 1
17             error_probs[symbol_index] = errors / trials
18     return error_probs

```

Listing 15: Monte Carlo estimation of sub-channel error probabilities for the q-ary symmetric channel.

D. Higher Alphabet BLER for $q \in \{2, 3, 4, 5, 6, 7\}$

The following plots show BLER vs. number of information symbols K for each tested alphabet size ($N = 256$, $p = 0.1$, 50 trials, construction via 100 Monte Carlo trials). As discussed in Section 6, these plots are not a meaningful comparison across alphabet sizes since K is measured in symbols rather than bits, which inherently favors larger alphabets. They are included here purely as an initial verification that the q -ary pipeline was functioning correctly and that average channel capacity increased with q , as expected from theory.

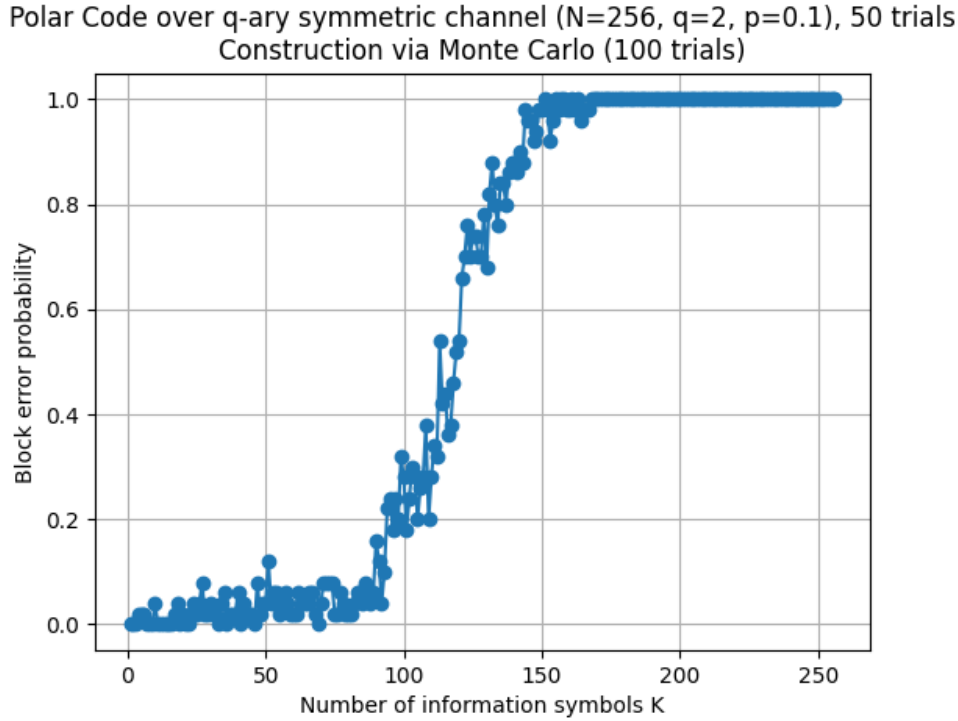


Figure 7: BLER vs. K for $q = 2$ ($N = 256$, $p = 0.1$).

Polar Code over q-ary symmetric channel ($N=256$, $q=3$, $p=0.1$), 50 trials
Construction via Monte Carlo (100 trials)

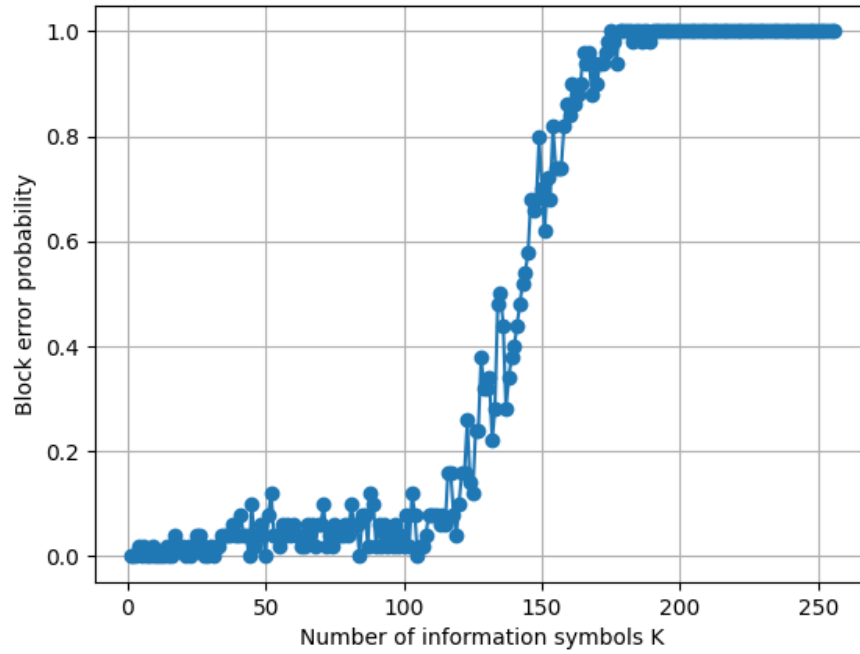


Figure 8: BLER vs. K for $q = 3$ ($N = 256$, $p = 0.1$).

Polar Code over q-ary symmetric channel ($N=256$, $q=4$, $p=0.1$), 50 trials
Construction via Monte Carlo (100 trials)

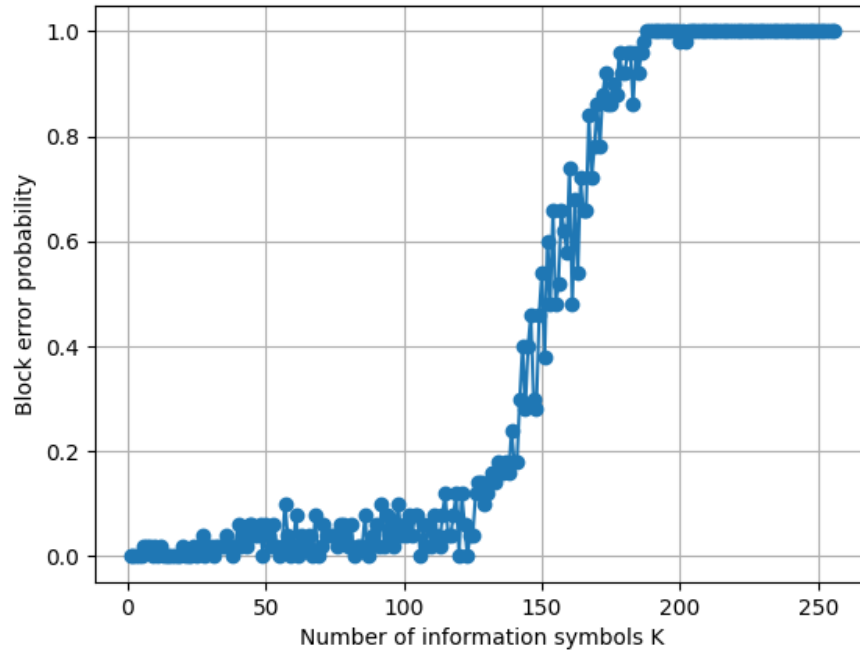


Figure 9: BLER vs. K for $q = 4$ ($N = 256$, $p = 0.1$).

Polar Code over q -ary symmetric channel ($N=256$, $q=5$, $p=0.1$), 50 trials
Construction via Monte Carlo (100 trials)

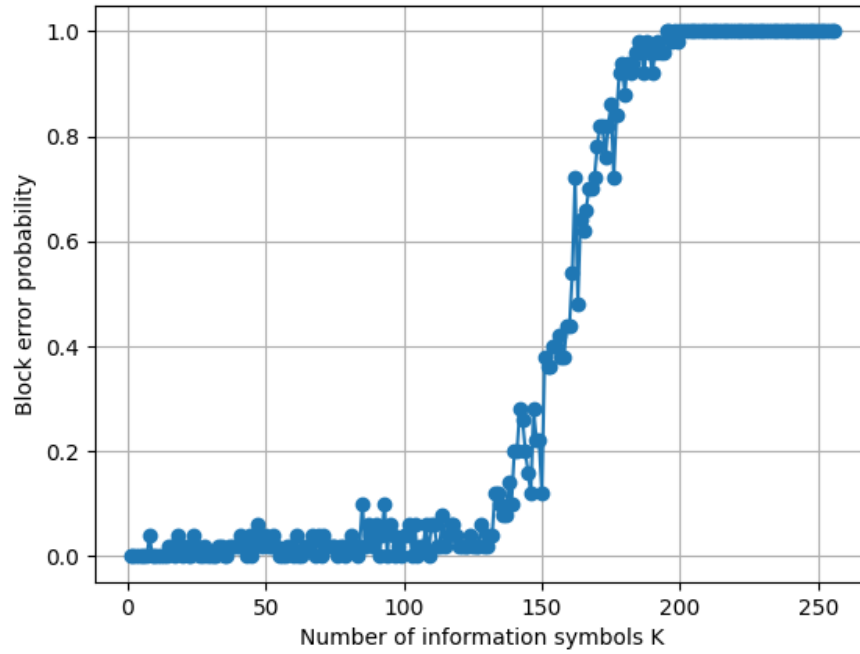


Figure 10: BLER vs. K for $q = 5$ ($N = 256$, $p = 0.1$).

Polar Code over q -ary symmetric channel ($N=256$, $q=6$, $p=0.1$), 50 trials
Construction via Monte Carlo (100 trials)

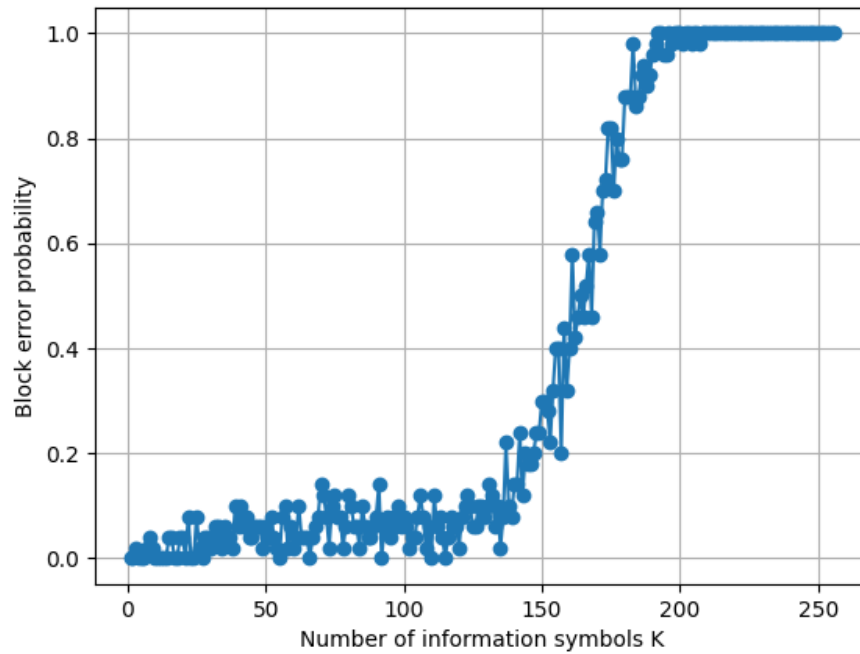


Figure 11: BLER vs. K for $q = 6$ ($N = 256$, $p = 0.1$).

Polar Code over q-ary symmetric channel ($N=256$, $q=7$, $p=0.1$), 50 trials
Construction via Monte Carlo (100 trials)

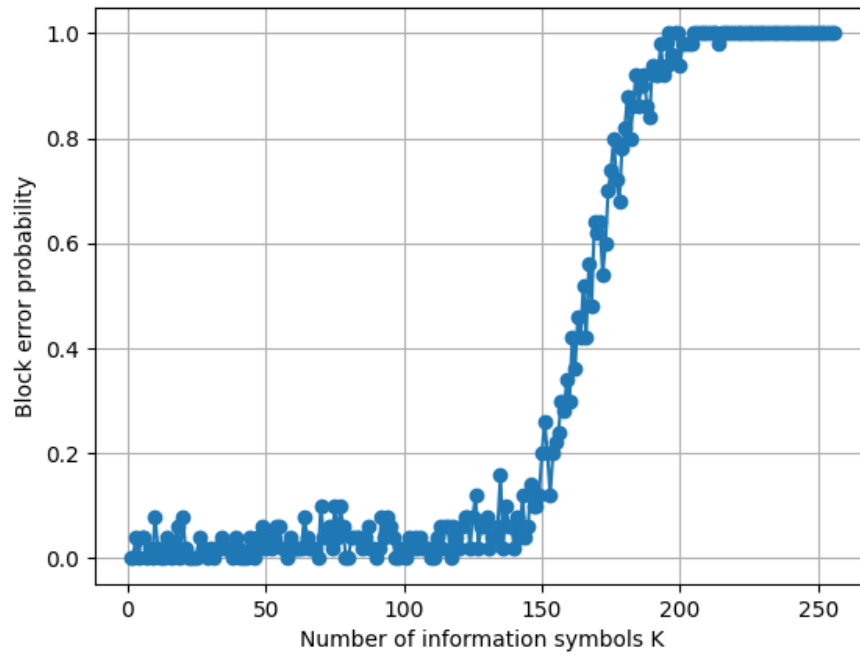


Figure 12: BLER vs. K for $q = 7$ ($N = 256$, $p = 0.1$).